

Refactorización del sistema de excepciones en OpenMarkov

Diseño, motivación e implementación

CISIAD — UNED

Marzo 2026

Índice

1. Introducción

OpenMarkov es una herramienta de modelado con redes probabilísticas compuesta por aproximadamente veinte módulos Maven y varios miles de clases Java. Antes de la refactorización descrita en este documento, el proyecto definía **61 clases de excepción propias**. La mayor parte de ellas eran excepciones *checked* (comprobadas), extendían `Exception` directamente y carecían de cualquier clase base común dentro del dominio de la aplicación.

Esta situación generaba varios problemas prácticos: métodos de la interfaz gráfica declaraban en su firma excepciones de bajo nivel que nunca podían manejar, el compilador no podía verificar la exhaustividad de los bloques `catch` sobre excepciones con múltiples variantes, y añadir una nueva acción en la GUI requería copiar decenas de líneas de código de gestión de errores sin ningún valor semántico.

Este informe documenta la refactorización completa del sistema de excepciones: los problemas identificados, las decisiones de diseño tomadas y su justificación, los cambios concretos aplicados al código y el impacto cuantitativo resultante.

2. Conceptos previos

2.1. Excepciones comprobadas y no comprobadas

Java distingue dos categorías de excepción:

Comprobada (*checked*) El compilador obliga al código que invoca un método a gestionar explícitamente la excepción: capturarla (`catch`) o propagarla (`throws`). Heredan de `Exception` pero no de `RuntimeException`.

No comprobada (*unchecked*) El compilador no impone ninguna obligación al llamador. Heredan de `RuntimeException`.

La elección tiene consecuencias arquitectónicas importantes. Una excepción comprobada que ninguna capa intermedia puede resolver *contamina* la firma de todos los métodos por los que transita, forzando cláusulas `throws` que no añaden información semántica. El principio generalmente aceptado es:

- Usar **checked** cuando el llamador dispone de una acción correctiva concreta y razonable (abrir otro fichero, retirar la evidencia conflictiva).
- Usar **unchecked** cuando el error no es recuperable en la capa llamante o indica un problema de programación.

2.2. Clases selladas (*sealed classes*)

Introducidas en Java 17, las clases selladas permiten declarar exactamente qué subclases están permitidas:

```

1 public abstract sealed class MiExcepcion extends
    RuntimeException
2     permits MiExcepcion.CasoA, MiExcepcion.CasoB {
3
4     public static final class CasoA extends MiExcepcion { ... }
5     public static final class CasoB extends MiExcepcion { ... }
6 }

```

Listing 1: Sintaxis de clase sellada

Las ventajas son:

- El compilador garantiza exhaustividad en expresiones `switch` sobre el tipo.
- Es imposible añadir un subtipo accidentalmente desde otro módulo.
- Los subtipos posibles quedan documentados en el código con nombre propio y campos tipados, sin depender de comentarios o convenciones informales.

2.3. Arquitectura en capas de OpenMarkov

OpenMarkov sigue una arquitectura en capas donde cada módulo depende de los inferiores pero no al revés:

Nivel	Módulo
4	org.openmarkov.gui
3	org.openmarkov.inference, learning.*
2	org.openmarkov.io.*
1	org.openmarkov.core

Cuando un tipo concreto de una capa inferior aparece en la firma de un método de una capa superior se produce una **fuga de abstracción**: la capa superior queda acoplada a un detalle de implementación de la inferior, dificultando el mantenimiento y los cambios futuros.

2.4. Sistema de localización: IBundledOpenMarkovException

OpenMarkov incluye un sistema de mensajes de error localizados (i18n) basado en ficheros `.properties`. La interfaz `IBundledOpenMarkovException` actúa de puente entre las clases de excepción y dicho sistema:

```

1 public interface IBundledOpenMarkovException extends
    IOpenMarkovException {
2
3     default String getExceptionTitle() {
4         // Deriva el título del nombre simple de la clase,
5         // insertando espacios antes de las mayúsculas:
6         // "WriterException" -> "Writer Exception"
7         return ClassLocalizable.titleFromClassName(this.getClass());
8     }
9 }

```

```

8      }
9
10     default String getExceptionMessage() {
11         // Busca la clave <NombreClase>.message en el .
           properties
12         // correspondiente al módulo donde vive la excepción.
13         return ClassLocalizable.messageFromClass(this.getClass()
           );
14     }
15
16     static String toString(IBundledOpenMarkovException ex) {
17         return ex.getExceptionTitle() + ": " + ex.
           getExceptionMessage();
18     }
19 }

```

Listing 2: Interfaz de localización de excepciones (extracto)

La convención del proyecto es que **toda excepción propia de OpenMarkov** implemente esta interfaz, garantizando que los diálogos de error de la GUI siempre dispongan de un título y un mensaje localizados sin código especial en cada punto de captura.

3. Estado anterior: problemas identificados

3.1. Ausencia de base común para excepciones de dominio

Antes de la refactorización, las excepciones propias del proyecto extendían directamente `Exception` o `RuntimeException` sin ningún tipo intermedio propio. Esta situación impedía:

- Capturar cualquier excepción de dominio de OpenMarkov en un único `catch` para tratamiento uniforme.
- Distinguir en tiempo de compilación entre una excepción de dominio y una excepción del sistema (`NullPointerException`, `IOException`, etc.).
- Garantizar que todas las excepciones implementaran `IBundledOpenMarkovException`: sin clase base que la implementara, la presencia de la interfaz dependía de la disciplina del programador en cada clase individual.

3.2. Excepciones comprobadas irrecuperables

Varios tipos de excepción estaban declarados como *checked* a pesar de representar condiciones de las que ninguna capa intermedia podía recuperarse. El propio código lo reconocía con comentarios `TODO`:

- `NonProjectablePotentialException`: «*Almost every exception is wrapped into `UnreachableException` or shown with `JOptionPane`*»

- `NotEvaluableNetworkException`: «*Only caught to show a generic message*»
- `UnexpectedInferenceException`: «*Catches are always wrapped with `UnreachableException`*»
- `InvalidNetworkTypeException`: «*Always wrapped in `UnreachableException`*»
- `CannotNormalizePotentialException`: «*Does this really happen in the GUI?*»

El efecto era una contaminación en cascada: **más de 40 cláusulas `throws`** en firmas de métodos de la GUI, el motor de inferencia y los módulos de aprendizaje, ninguna de las cuales añadía información semántica útil. Métodos de Swing como `actionPerformed`, que por contrato del *framework* no deberían declarar excepciones, acumulaban cláusulas `throws` que el compilador toleraba sin que tuvieran ningún significado en ese contexto.

3.3. Fuga de `SAXException` hasta la GUI

El método `MainPanelListenerAssistant.openNetwork()` declaraba `throws SAXException`. `SAXException` pertenece a la biblioteca estándar de procesamiento XML (JAXP/SAX) y es completamente ajena al dominio de redes probabilísticas. Su presencia en la firma de un método de la capa de presentación representa una fuga de abstracción: la GUI estaba acoplada a un detalle de implementación del módulo de lectura de ficheros.

3.4. Jerarquía plana con variantes implícitas

`NonProjectablePotentialException` agrupaba ocho causas de error distintas sin ninguna estructura formal. Los bloques `catch` debían recurrir a `instanceof` o a campos de texto para distinguirlas, lo que:

- Requería que el programador conociera de memoria los subtipos informales.
- No era verificado por el compilador.
- Hacía que el código de manejo de errores fuera frágil ante la adición de nuevos casos.

3.5. Bloques `try/catch` repetitivos en la GUI

El método `actionPerformed` de `MainPanelListenerAssistant` contenía aproximadamente **34 bloques** con la estructura:

```

1 try {
2     accionConcreta();
3 } catch (ExcepcionComprobada1 ex) {
4     throw new UnrecoverableException(ex);
5 } catch (ExcepcionComprobada2 ex) {
6     throw new UnrecoverableException(ex);
7 }
```

Listing 3: Patrón repetitivo antes de la refactorización

Este código no añadía ninguna lógica: simplemente envolvía toda excepción *checked* en `UnrecoverableException`. Era ruido puro, propenso a errores al añadir nuevas acciones y difícil de leer.

3.6. Antipatrón `catch → null` en `FormatManager`

El método de instanciación de plugins de `FormatManager` capturaba las excepciones de reflexión y devolvía `null` sin registrar nada:

```
1 .map(readerClass -> {
2     try {
3         return readerClass.getDeclaredConstructor().newInstance
4             ();
5     } catch (InstantiationException | NoSuchMethodException |
6         ...) {
7         return null; // plugin mal configurado: fallo
8             silencioso
9     }
10 })
```

Listing 4: Antipatrón original en `FormatManager`

Un plugin mal configurado fallaba silenciosamente en este punto y provocaba un `NullPointerException` sin traza útil en un punto posterior y completamente distinto del origen del problema.

Además, el método `getProbNetReaderInstanceFor()` usaba `.findFirst().get()` sobre el `Optional` resultante de la búsqueda de lector. En ausencia de lector, este código lanzaba `NoSuchElementException` en lugar de devolver `null`, convirtiendo el `if (reader == null)` del llamador en código muerto e inalcanzable.

3.7. Excepciones de navegación de casos de evidencia

`EvidenceManager` declaraba como *checked* `ThereIsNoPreviousEvidenceCaseException` y `ThereIsNoNextEvidenceCaseException`. Sin embargo, el panel de controles ya desactivaba los botones correspondientes antes de que pudieran provocar esas condiciones. Las excepciones eran código muerto que se propagaba innecesariamente a los módulos de inferencia y aprendizaje.

4. Diseño de la nueva jerarquía

4.1. Criterio para la clasificación `checked` / `unchecked`

El criterio aplicado es semántico y se resume en la siguiente pregunta:

¿Existe en el llamador una acción correctiva concreta y razonable cuando se produce este error?

- Si la respuesta es **sí** (mostrar un diálogo y ofrecer abrir otro fichero, retirar la evidencia conflictiva, etc.): la excepción debe ser **checked**, para que el compilador obligue al llamador a considerar ese escenario.
- Si la respuesta es **no** (el llamador solo puede registrar el error y terminar, o la condición no debería ocurrir bajo un diseño correcto): la excepción debe ser **unchecked**, para no contaminar las firmas.

Aplicando este criterio, las cinco excepciones del §?? pasaron a ser *unchecked*. El resto mantuvieron su carácter *checked*.

4.2. Clases base abstractas: OpenMarkovRuntimeException y OpenMarkovException

Se crearon dos clases base abstractas simétricas para las dos categorías:

```

1 public abstract class OpenMarkovRuntimeException extends
    RuntimeException
2     implements IBundledOpenMarkovException {
3     protected OpenMarkovRuntimeException() {}
4     protected OpenMarkovRuntimeException(Throwable cause) {
        super(cause); }
5 }
```

Listing 5: OpenMarkovRuntimeException — base *unchecked*

```

1 public abstract class OpenMarkovException extends Exception
2     implements IBundledOpenMarkovException {
3     protected OpenMarkovException() {}
4     protected OpenMarkovException(Throwable cause) { super(cause
        ); }
5     protected OpenMarkovException(String message) { super(
        message); }
6     protected OpenMarkovException(String message, Throwable
        cause) {
7         super(message, cause);
8     }
9 }
```

Listing 6: OpenMarkovException — base *checked*

OpenMarkovException define cuatro constructores (frente a dos en OpenMarkovRuntimeException) porque las excepciones *checked* existentes que se migraban usaban también los constructores con `String`.

Al implementar `IBundledOpenMarkovException`, ambas clases base garantizan que todas sus subclases hereden automáticamente `getExceptionTitle()` y `getExceptionMessage()`, sin necesidad de añadir la interfaz de forma individual en cada subclase.

4.3. Agrupación semántica: `UserInputException`

Dentro de la jerarquía *checked*, se identificó un grupo de excepciones con una propiedad común: todas son consecuencia directa de lo que el usuario proporciona al sistema, y la acción correctiva en todos los casos es mostrar un mensaje informativo. Las excepciones del grupo son:

Excepción	Causa
<code>ParserException</code>	Fichero abierto mal formado o versión no reconocida
<code>WriterException</code>	Fallo al escribir el fichero de red
<code>IncompatibleEvidenceException</code>	Evidencia inconsistente con la red
<code>EmptyDatabaseException</code>	Base de datos sin casos para aprendizaje
<code>ParsingSourceException</code>	Expresión fuente con error sintáctico
<code>NoReaderForFileException</code>	Ningún plugin puede leer el fichero indicado
<code>NoReaderForExtension</code>	Sin lector para la extensión dada
<code>NoWriterForExtensionException</code>	Sin escritor para la extensión dada

Se creó `UserInputException` `extends OpenMarkovException` como clase intermedia abstracta y estas ocho excepciones pasaron a extenderla. La motivación es doble:

1. **Captura uniforme en la GUI.** El código de presentación puede escribir:

```
1 } catch (UserInputException ex) {  
2     showErrorDialog(ex.getExceptionTitle(),  
3                     ex.getExceptionMessage());  
4 }
```

Listing 7: Captura uniforme de errores de usuario

sin enumerar cada subtipo. La lista de subtipos puede crecer en el futuro sin modificar el código de la GUI.

2. **Documentación estructural.** La pertenencia a `UserInputException` comunica a quien mantiene el código que el error es tratable a nivel de usuario, en contraposición a errores internos como `DoEditException` o `CostEffectivenessException`.

4.4. Excepciones *checked* fuera de `UserInputException`

Las excepciones *checked* que no se incluyeron en `UserInputException` son aquellas cuyo tratamiento no consiste simplemente en mostrar un mensaje:

`DoEditException` / `ConstraintViolatedException` Señalizan fallos en operaciones de edición de la red (undo/redo) o violaciones de restricciones del modelo. Tienen puntos de captura específicos con lógica de tratamiento propia.

`CostEffectivenessException` Indica configuraciones incorrectas en el análisis coste-efectividad, con validación específica.

`PotentialOperationException` Fallo en operaciones matemáticas sobre potenciales, con tratamiento en el motor de inferencia.

Excepciones de intervalo Restricciones de integridad matemática con validación en el modelo.

4.5. Jerarquías selladas para excepciones con múltiples variantes

Para las excepciones con múltiples causas distintas se eligió el patrón de **clase sellada con subclases internas estáticas**. El caso más representativo es `NonProjectablePotentialException`:

```
1 public abstract sealed class NonProjectablePotentialException
2     extends OpenMarkovRuntimeException {
3
4     public static final class SuperValueMustBeSumOrProduct
5         extends NonProjectablePotentialException { ... }
6
7     public static final class PotentialCannotBeConvertedToATable
8         extends NonProjectablePotentialException { ... }
9
10    public static final class MissingVariableInEvidence
11        extends NonProjectablePotentialException {
12        public MissingVariableInEvidence(Variable variable) {
13            ... }
14        public final Variable variable;
15    }
16    // ... cinco subtipos más
17 }
```

Listing 8: Jerarquía sellada de `NonProjectablePotentialException` (extracto)

El mismo patrón se aplicó a `NotEvaluableNetworkException` (3 subtipos), `InvalidNetworkTypeException` (2 subtipos), `UnexpectedInferenceException` (1 subtipo), `WriterException` (5 subtipos), `ParsingSourceException` (1 subtipo), `CostEffectivenessException` (4 subtipos) y `PotentialOperationException` (1 subtipo).

4.6. Exclusión de `UnreachableException` y `UnrecoverableException`

Estas dos excepciones tienen naturaleza distinta y fueron excluidas deliberadamente de las jerarquías de dominio:

`UnreachableException` Señaliza código que, por diseño del sistema, nunca debería ejecutarse. Es equivalente a `assert false`: no es un error de dominio sino la violación de una invariante del programa. Incluirla en `OpenMarkovRuntimeException` habría confundido errores de dominio con errores de programación en los bloques `catch`.

`UnrecoverableException` Es infraestructura de la GUI: envuelve cualquier excepción que alcanza el *event dispatch thread* de Swing sin haber sido tratada. Su alcance es transversal, no específico del dominio.

4.7. Preservación de nombres existentes

Una opción considerada fue renombrar excepciones para encajarlas en una nueva taxonomía (por ejemplo, `NonProjectablePotentialException` $\rightarrow$ `PotentialComputationException`). Esta opción se descartó porque:

- Los nombres originales son vocabulario establecido entre los desarrolladores del proyecto y tienen significado semántico preciso en el dominio.
- El cambio de nombre habría requerido actualizar cientos de ficheros, introduciendo ruido en el historial de versiones sin beneficio funcional.
- La taxonomía deseada se expresa igualmente bien mediante herencia.

5. Cambios implementados

5.1. Conversión a no comprobadas de cinco excepciones de dominio

Las cinco excepciones identificadas en § ?? se convirtieron a `extends OpenMarkovRuntimeException`. Esto eliminó **más de 40 cláusulas** `throws` a través de 53 ficheros en cuatro módulos (`core`, `gui`, `inference`, `learning.algorithm`).

5.2. Jerarquías selladas

`NonProjectablePotentialException` y otras seis excepciones se convirtieron en clases selladas con subtipos internos tipados (véase § ??). Los bloques `catch` que anteriormente recurrían a `instanceof` pueden ahora trabajar con tipos concretos verificados en tiempo de compilación.

5.3. Patrón `UIAction` en la GUI

Se introdujo la interfaz funcional `UIAction` y el método auxiliar `executeUIAction` en `MainPanelListenerAssistant`:

```
1 @FunctionalInterface
2 private interface UIAction {
3     void execute() throws Exception;
4 }
5
6 private void executeUIAction(UIAction action) {
7     try {
8         action.execute();
9     } catch (RuntimeException ex) {
10         throw ex; // deja pasar unchecked
11                 sin envolver
12     } catch (Exception ex) {
```

```

12         throw new UnrecoverableException(ex);
13     }
14 }

```

Listing 9: Interfaz funcional `UIAction` y método auxiliar

Los 34 bloques `try/catch` repetitivos se reemplazaron por llamadas de una sola línea:

```

1 case ActionCommands.OPEN_NETWORK ->
2     executeUIAction(() -> openNetwork());
3
4 case ActionCommands.COMPILE_NETWORK -> executeUIAction(() -> {
5     compileNetwork();
6     updateMenus();
7 });

```

Listing 10: Uso del patrón `UIAction` en `actionPerformed`

Se dejaron fuera del patrón dos casos con lógica especial: `CHANGE_WORKING_MODE` (tiene su propia lógica de confirmación de cambios sin guardar) y `COST_EFFECTIVENESS_DETERMINISTIC` (lanza `UnreachableException`, no una excepción comprobada).

5.4. Encapsulación de `SAXException`

Se añadió un bloque `catch(SAXException)` en el interior de `openNetwork()` que convierte la excepción en `ParserException.BadlyStructuredFile` antes de que escape del método. La firma de `openNetwork()` ya no menciona `SAXException` y la GUI queda completamente aislada de la tecnología de *parsing* XML.

5.5. Eliminación de excepciones de navegación

Se eliminaron `ThereIsNoPreviousEvidenceCaseException` y `ThereIsNoNextEvidenceCaseException`. Los bloques que las lanzaban se sustituyeron por `UnreachableException`:

```

1 if (!(this.currentCase > 0)) {
2     throw new UnreachableException(new IllegalStateException(
3         "Go-to-previous button should have been disabled " +
4         "when at first evidence case"));
5 }

```

Listing 11: Guarda con `UnreachableException` en `goToPreviousEvidenceCase`

`UnreachableException` comunica explícitamente a los futuros mantenedores que esta rama es imposible bajo las invariantes actuales, a diferencia de un comentario que puede quedar desfasado.

5.6. Corrección del antipatrón en `FormatManager`

Se realizaron dos correcciones:

1. **Instanciación de plugins:** el bloque `catch` que devolvía `null` fue sustituido por `throw new UnreachableException(...)`, que incluye el nombre de la clase y la causa original.
2. **Búsqueda de lector:** `.findFirst().get()` fue sustituido por `.findFirst().orElse(null)`, restaurando el comportamiento pretendido:

```
1 // ANTES      lanza NoSuchElementException; el null-check es
   código muerto
2 return this.readerInstances.stream()
3     .filter(plugin -> ...)
4     .findFirst().get();
5
6 // DESPUÉS   retorna null; el llamador lanza
   NoReaderForFileException
7 return this.readerInstances.stream()
8     .filter(plugin -> ...)
9     .findFirst().orElse(null);
```

Listing 12: Corrección en `getProbNetReaderInstanceFor()`

5.7. Creación de `OpenMarkovException` y migración de excepciones *checked*

Se creó la clase abstracta `OpenMarkovException` (véase §??) y las excepciones *checked* que extendían `Exception` directamente se migraron a extenderla. El proceso afectó a **42 ficheros** en siete módulos. La migración eliminó automáticamente las cláusulas `implements IBundledOpenMarkovException` redundantes de cada una de ellas.

5.8. Creación de `UserInputException`

Se creó `UserInputException` `extends OpenMarkovException` y las ocho excepciones descritas en §?? pasaron a extenderla. El cambio es **totalmente compatible hacia atrás**: cualquier código que capture `OpenMarkovException` o un subtipo concreto continúa funcionando sin modificaciones.

6. Jerarquía resultante

Las figuras ?? y ?? muestran la jerarquía de excepciones tras la refactorización completa. Las clases sombreadas en verde (`OpenMarkovException`, `UserInputException`) son nuevas. Las sombreadas en azul (`OpenMarkovRuntimeException` y su rama) constituyen la jerarquía *unchecked*. Las sombreadas en rojo son clases de la biblioteca estándar de Java.

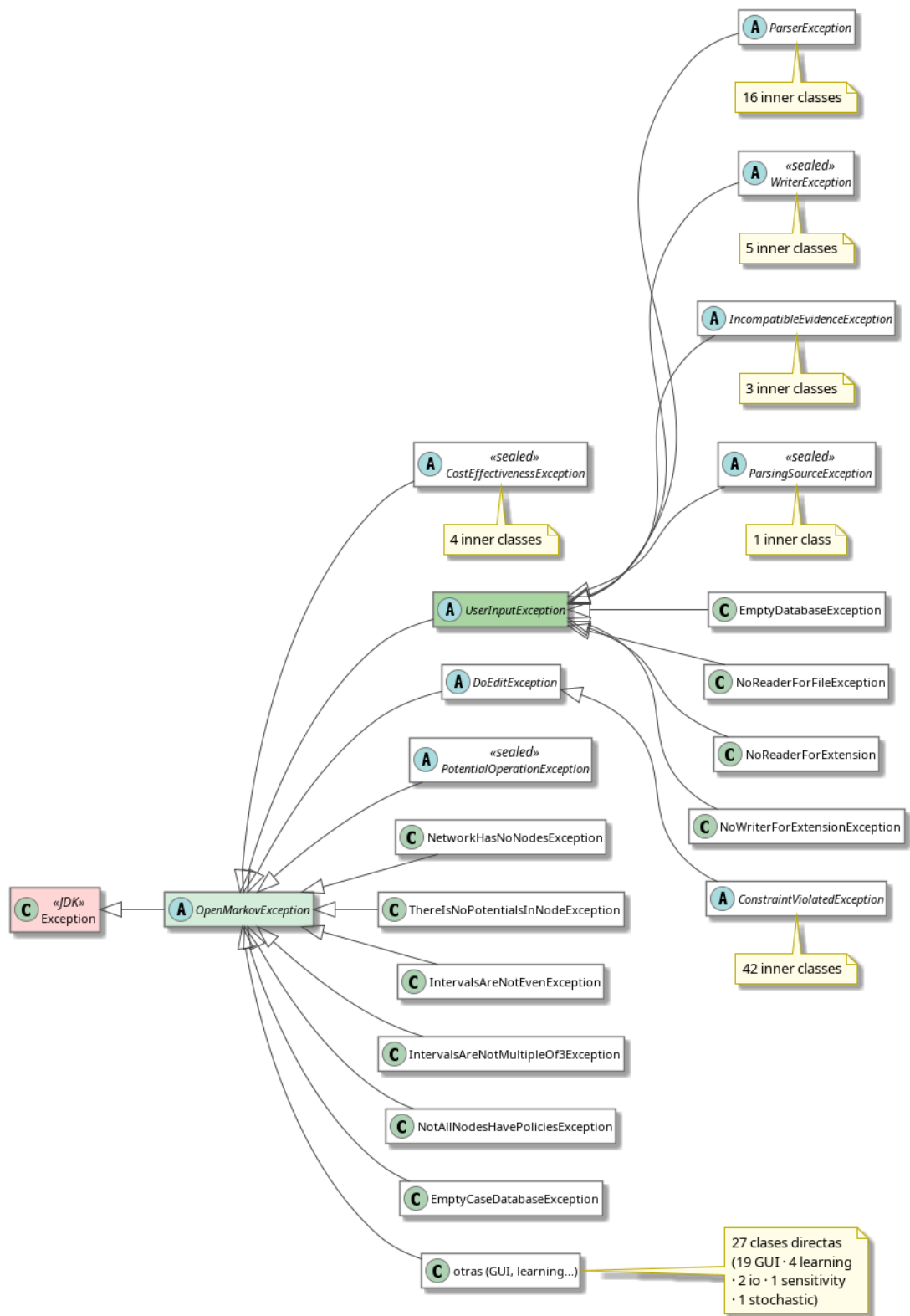


Figura 1: Jerarquía *checked* bajo `OpenMarkovException`. El número de *inner classes* se indica en notas. Las clases en verde son nuevas.

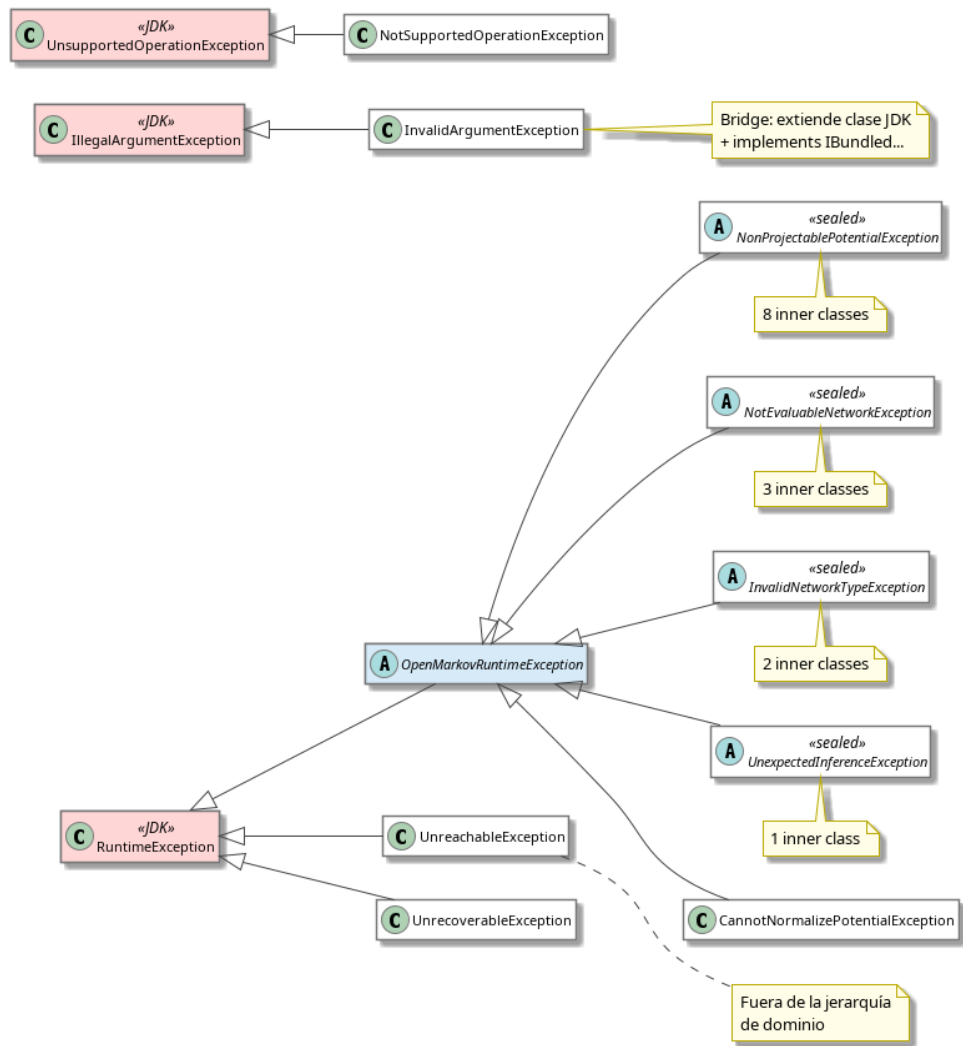


Figura 2: Jerarquía *unchecked* bajo `OpenMarkovRuntimeException` e infraestructura de gestión de errores.

7. Impacto cuantitativo

Cambio	Ficheros	Elementos	Efecto
Conversión de 5 excepciones a <i>unchecked</i>	53	−40 throws	Elimina contaminación en cascada
Jerarquías selladas (7 excepciones)	8	+24 subtipos tipados	Exhaustividad en compilación
Patrón <code>UIAction</code> en GUI	1	−34 bloques <code>try/catch</code>	Elimina código repetitivo
Encapsulación de <code>SAXException</code>	1	−1 throws <code>SAXException</code>	Elimina fuga de abstracción
<code>OpenMarkovRuntimeException</code>	6	+1 base; +5 herencias	Base común <i>unchecked</i>
Eliminación de excepciones de navegación	3	−2 clases	Elimina código muerto
Corrección <code>FormatManager</code>	1	2 bugs corregidos	Fallos dejan de ser silenciosos
<code>OpenMarkovException</code>	1	+1 base <i>checked</i>	Base común <i>checked</i>
Migración de 42 excepciones <i>checked</i>	42	−42 implements redundantes	Coherencia de jerarquía
<code>UserInputException</code>	9	+1 intermedia; +8 herencias	Agrupación semántica
	15		
Total	~125	net ≈ −115	

Tabla 1: Resumen cuantitativo del refactoring

8. Conclusiones

La refactorización del sistema de excepciones de OpenMarkov produjo las siguientes mejoras concretas:

Legibilidad Los métodos expresan su lógica sin distracciones de manejo de errores que no aportan información. La eliminación de 40 cláusulas `throws` en cascada es el ejemplo más visible.

Coherencia del modelo de errores Todas las excepciones de dominio comparten una raíz común (`OpenMarkovException` o `OpenMarkovRuntimeException`) que garantiza la implementación de `IBundledOpenMarkovException` sin depender de la disciplina individual del programador.

Seguridad estática Las jerarquías selladas permiten que el compilador detecte bloques `switch` no exhaustivos sobre las variantes de las excepciones más complejas.

Expresividad de intención `UserInputException` comunica estructuralmente qué errores son orientados al usuario y cuáles son internos. `UnreachableException` documenta en el código mismo que una rama es imposible según las invariantes del sistema, de forma más fiable que un comentario.

Mantenibilidad Añadir una nueva acción en `actionPerformed` ya no requiere recordar el patrón `try/catch` repetitivo. Añadir una nueva excepción orientada al usuario solo requiere extender `UserInputException`.

Separación de capas La GUI ya no conoce `SAXException` ni ningún otro detalle de implementación de la capa de E/S.

Diagnóstico de errores Los fallos en la instanciación de plugins de `FormatManager` incluyen ahora el nombre de la clase problemática y la causa original en lugar de fallar silenciosamente varios pasos después.

Todos los tests existentes pasan tras la refactorización, lo que proporciona confianza en que no se han introducido regresiones en el comportamiento observable.